

Name of the Student	P Surendra Kumar Reddy
Reg. No	192425224
GitHub Link	https://github.com/suri307/MLA0403-DEEP-LEARNING/tree/main/ASSESSMENTS

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Model

COURSE NAME: Deep Learning
SLOT :

COURSE CODE: MLA04

COURSE OUTCOME COVERED

CO 4: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. BL-6

Course Outcome Weightage: 10%
Assessment Tool Weightage: 30%

Duration: 90 minutes
Mark : 25

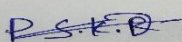
Code of Conduct:

I P Surendra Kumar Reddy(192425224) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Name of the student: P Surendra Kumar Reddy



Criteria	Problem Understanding & Requirement Analysis (2)	Model Architecture & Design (2.5)	Application of Deep Learning Techniques (2.5)	Model Development & Implementation Strategy (2)	Performance Analysis & Justification (1)	Total (10)
Weightage	20 %	30%	25%	15 %	10 %	100 %
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

1. **Design and develop a model for a hospital** has only **2,000 labeled X-ray images** for classifying images into normal and abnormal categories. A pre-trained ResNet model is available. **Design a CNN-based transfer learning model** by identifying which layers should be frozen, which layer should be replaced, and how the model should be trained.
2. An agricultural organization wants to identify **five types of plant diseases** from leaf images. Since the dataset contains limited training samples, the organization wants to reuse features efficiently. **Design a DenseNet-based classification model** and explain how dense connections can help the model reuse features during classification.
3. A university wants to predict a student's future academic performance using their **weekly attendance, assignment marks, internal assessment marks, and previous examination scores** collected over several weeks. **Design an LSTM-based model** that can process this sequential information and predict the student's performance category.
4. An organization wants to classify customer reviews as **positive, negative, or neutral**. Since the meaning of a word may depend on both the words appearing before and after it, **design a Bidirectional RNN model** for this application. Identify the major processing stages from input text to final sentiment prediction.
5. A software company wants to develop a system that translates **English sentences into Hindi**. The length of the input and output sentences may differ, and the system needs to understand the complete input sentence before generating the translated sentence. **Design an Encoder–Decoder model** and describe the role of the encoder, decoder, hidden representation, and output layer in the translation process.

Q1. ResNet-Based Transfer Learning for X-Ray Classification

Question:

A hospital has only 2,000 labeled X-ray images for classifying images into **normal** and **abnormal** categories. A pre-trained ResNet model is available. Design a CNN-based transfer learning model by identifying which layers should be frozen, which layer should be replaced, and how the model should be trained.

Solution:

1. Problem Understanding

The hospital has only **2,000 labeled X-ray images**, which is a relatively small dataset for training a deep CNN from scratch. Therefore, a **pre-trained ResNet model** can be used through **transfer learning**.

The model will perform **binary classification**:

- **Class 0:** Normal
- **Class 1:** Abnormal

Transfer learning allows the model to reuse features learned from a large dataset instead of learning all features from the beginning.

2. Model Architecture

The proposed architecture is:

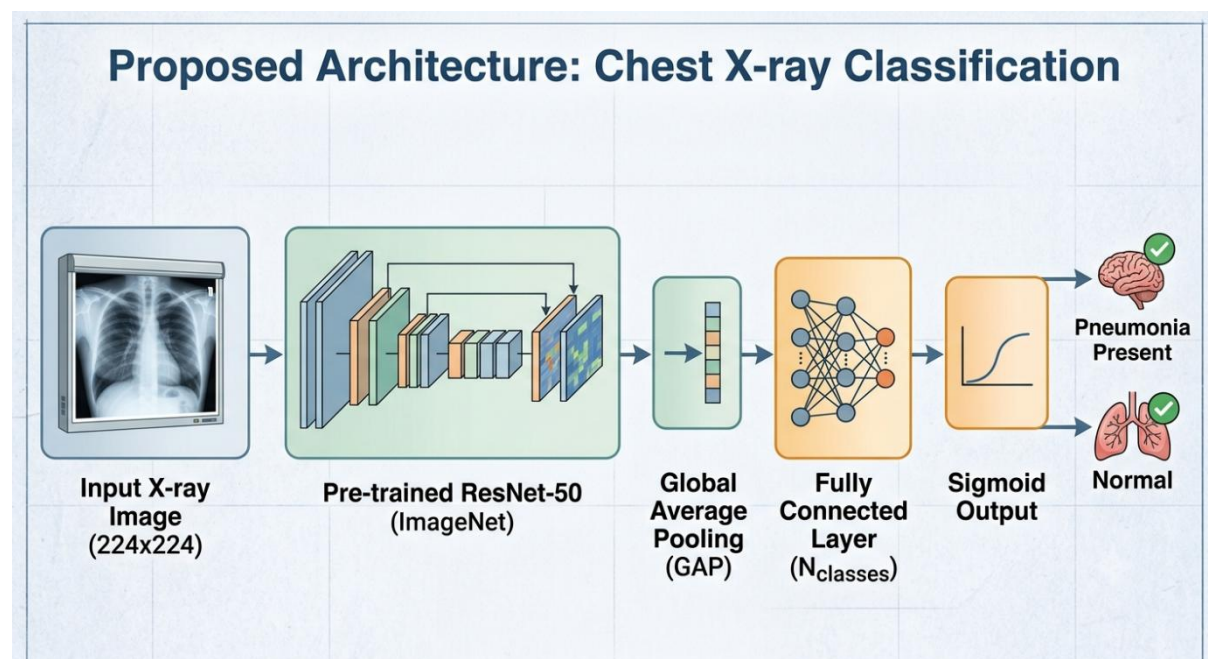


Figure 1: Model Architecture

A pre-trained **ResNet** is selected as the base CNN.

- The initial ResNet layers learn general features such as edges, shapes, and textures.
- The deeper layers learn more complex image features.

- Since the available X-ray dataset is small, most of the pre-trained layers are initially kept frozen.
- The original classification layer is replaced with a new binary classification layer.

3. Frozen Layers

The **convolutional layers of the pre-trained ResNet** are frozen initially.

This means:

- Their learned weights are not updated during the first training stage.
- The model can reuse the already learned visual features.
- It reduces training time.
- It also reduces the risk of overfitting because only 2,000 images are available.

4. Replaced Layer

The original final classification layer of ResNet is removed and replaced with:

Fully Connected (Dense) Layer → 1 neuron → Sigmoid activation

The sigmoid output gives the probability of the X-ray being abnormal.

For example:

- Output close to **0** → **Normal**
- Output close to **1** → **Abnormal**

5. Training Strategy

The model can be trained in two stages.

Stage 1:

- Freeze the ResNet convolutional layers.
- Train only the newly added classification layer.
- Use **binary cross-entropy loss**.
- Use an optimizer such as **Adam**.
- Apply data augmentation such as rotation, flipping, and small zooms where medically appropriate.

Stage 2:

- Unfreeze some of the deeper ResNet layers.
- Fine-tune these layers using a **small learning rate**.
- Keep the earlier layers frozen because they contain general visual features.
- Monitor validation performance to prevent overfitting.

6. Performance Analysis

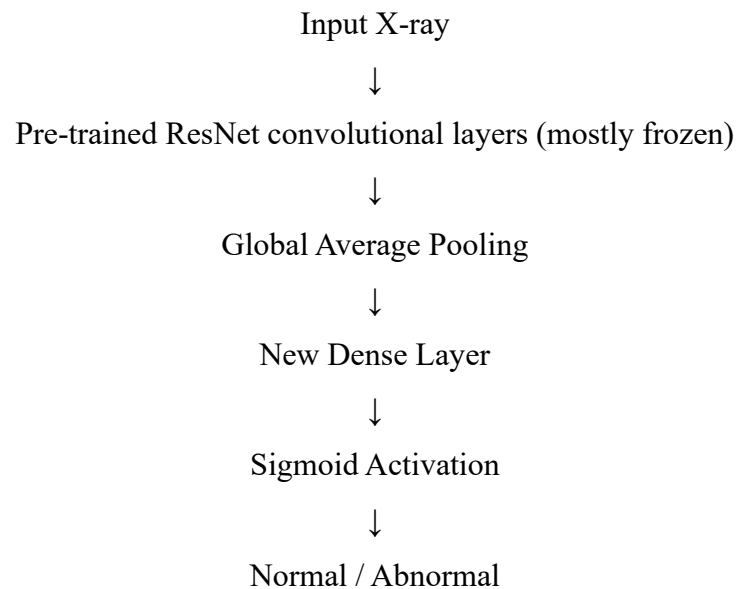
The model should be evaluated using:

- **Accuracy**
- **Precision**

- **Recall**
- **F1-score**
- **Confusion Matrix**

For medical X-ray classification, **recall is particularly important**, because missing an abnormal case can be more serious than incorrectly classifying a normal case.

7. Final Model



Conclusion

A **ResNet-based transfer learning model** is suitable because the hospital has only 2,000 labeled X-ray images. The pre-trained ResNet layers can be frozen to reuse learned features, while the original final classification layer is replaced with a **single-neuron sigmoid layer** for binary classification. After initial training, some deeper ResNet layers can be unfrozen and fine-tuned with a low learning rate to improve performance.

Q2. DenseNet-Based Plant Disease Classification

Question:

An agricultural organization wants to identify **five types of plant diseases from leaf images**. Since the dataset contains limited training samples, the organization wants to reuse features efficiently. Design a **DenseNet-based classification model** and explain how dense connections can help the model reuse features during classification.

Solution:

1. Problem Understanding

The objective is to classify plant leaf images into **five different disease categories**. Since the available training dataset is limited, training a deep CNN from scratch may lead to overfitting. A **DenseNet (Densely Connected Convolutional Network)** is suitable because each layer can directly receive information from earlier layers. This allows the network to **reuse previously learned features** and reduces unnecessary learning of the same features repeatedly.

2. Proposed Model Architecture

The proposed architecture is:

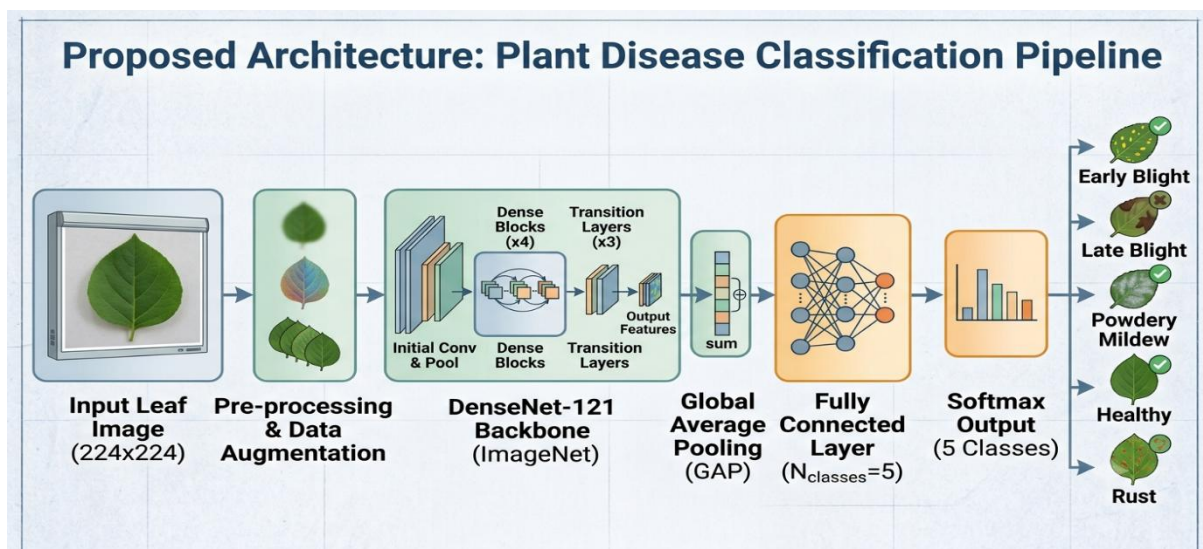


Figure 2: Proposed Architecture

The input leaf image is passed through the DenseNet feature-extraction layers. The final classification layer is designed with **five output neurons**, one for each plant disease category.

3. Dense Connections

The main feature of DenseNet is its **dense connectivity**.

In a normal CNN, a layer mainly receives the output of the previous layer:

Layer 1 → Layer 2 → Layer 3 → Layer 4

In DenseNet, each layer receives feature maps from **all preceding layers** within a dense block:

Layer 1 → Layer 2

Layer 1 + Layer 2 → Layer 3

Layer 1 + Layer 2 + Layer 3 → Layer 4

Therefore, earlier features can be directly reused by later layers.

4. Feature Reuse

Dense connections allow the model to reuse features such as:

- Edges
- Leaf shapes
- Textures
- Vein patterns
- Spots and lesions
- Disease-specific patterns

For example, an early layer may detect the **edges of a leaf**, while a deeper layer can reuse those edges together with texture and color-related features to identify a particular disease.

This improves feature propagation and makes the network more efficient.

5. Training Strategy

The DenseNet model can be trained using the following approach:

- Resize all leaf images to a common input size.
- Normalize the image pixel values.
- Apply data augmentation such as rotation, zooming, cropping, and horizontal flipping where appropriate.
- Use a pre-trained DenseNet model if available to benefit from transfer learning.
- Replace the original classification layer with a **5-class Softmax layer**.
- Initially train the new classification layer.
- Fine-tune selected deeper DenseNet layers using a small learning rate.
- Use **categorical cross-entropy loss** for the five-class classification problem.
- Use an optimizer such as **Adam**.

6. Classification Output

The final Softmax layer contains **5 neurons**.

Output	Disease Class
Class 1	Disease 1
Class 2	Disease 2
Class 3	Disease 3
Class 4	Disease 4
Class 5	Disease 5

The class with the highest Softmax probability is selected as the predicted disease.

7. Performance Evaluation

The model can be evaluated using:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-score**
- **Confusion Matrix**

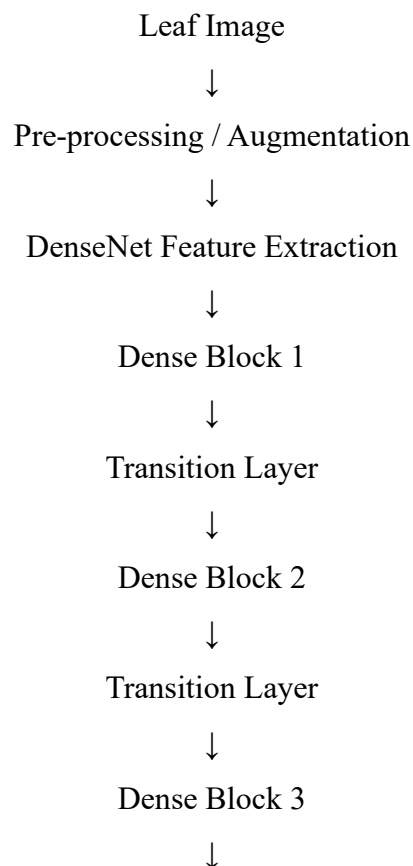
Since there are five disease classes, the confusion matrix can show which diseases are being confused with one another.

8. Advantages of DenseNet for This Problem

DenseNet is suitable for the limited dataset because:

1. **Feature reuse:** Earlier features are reused by later layers.
2. **Efficient learning:** The network avoids repeatedly learning similar features.
3. **Better feature propagation:** Information can pass directly through dense connections.
4. **Reduced redundancy:** Learned features can be shared among multiple layers.
5. **Useful for image classification:** It can learn detailed visual patterns required for plant disease identification.

9. Final Architecture



Global Average Pooling



Fully Connected Layer



Softmax (5 Outputs)



Predicted Plant Disease

Conclusion

A **DenseNet-based classification model** is appropriate for plant disease identification when training samples are limited. Its dense connections allow each layer to access features learned by previous layers, which promotes **feature reuse and efficient information propagation**. With a final **five-class Softmax layer**, the model can classify leaf images into the required five disease categories.

Q3. LSTM-Based Student Performance Prediction

Question:

A university wants to predict a student's future academic performance using their weekly attendance, assignment marks, internal assessment marks, and previous examination scores collected over several weeks. Design an **LSTM-based model** that can process this sequential information and predict the student's performance category.

Solution:

1. Problem Understanding

The student's academic information is collected **week by week**, so the data is sequential. The performance in one week may be related to performance in previous weeks.

An **LSTM (Long Short-Term Memory)** network is suitable because it can learn patterns from sequential data and remember important information from previous time steps.

The model can predict categories such as:

- **Low Performance**
- **Average Performance**
- **High Performance**

2. Input Data

For each week, the model receives four features:

1. Weekly attendance
2. Assignment marks
3. Internal assessment marks
4. Previous examination scores

For example:

Week 1 → Week 2 → Week 3 → ... → Week N

Each week is represented by a feature vector:

[Attendance, Assignment Marks, Internal Marks, Previous Exam Score]

3. Proposed Model Architecture

The proposed architecture is:

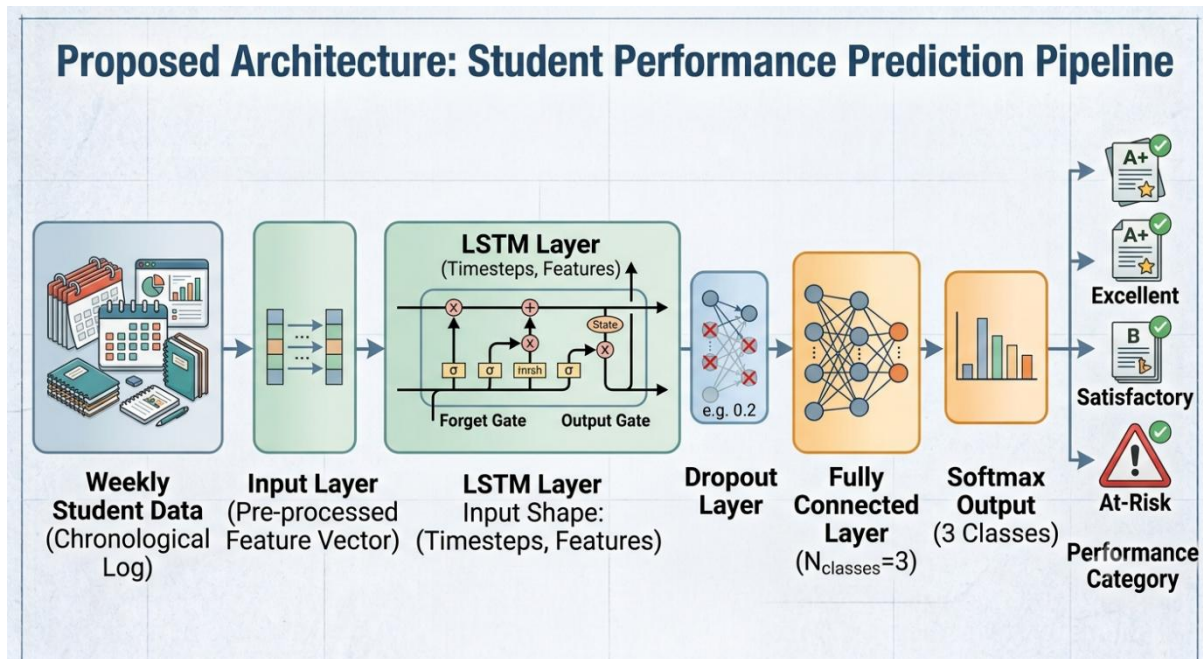


Figure 3: Proposed Architecture

The LSTM processes the weekly information in sequence and maintains useful information from earlier weeks.

4. Role of LSTM

The main advantage of LSTM is its ability to remember important information for a longer period.

For example, if a student had consistently low attendance during several previous weeks, this information may help predict their future academic performance.

LSTM uses internal gates to control information:

- **Forget gate:** Decides which old information should be removed.
- **Input gate:** Decides which new information should be stored.
- **Output gate:** Decides which information should be passed to the next stage.

This allows the model to focus on important patterns in the student's academic history.

5. Training Strategy

The training process can be performed as follows:

- Collect weekly student performance data.
- Arrange the data in chronological order.
- Normalize numerical features such as marks and attendance.
- Divide the data into training and testing sets.
- Feed sequences of weekly data into the LSTM.
- Use a **Softmax output layer** for the performance categories.

- Use **categorical cross-entropy loss** for multi-class classification.
- Use an optimizer such as **Adam**.
- Train the model for multiple epochs while monitoring validation performance.

6. Output Layer

If there are three performance categories, the final layer contains **3 neurons with Softmax activation**.

Output Performance Category

Class 1 Low

Class 2 Average

Class 3 High

The category with the highest probability becomes the predicted performance category.

7. Performance Evaluation

The model can be evaluated using:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-score**
- **Confusion Matrix**

These measures help determine whether the LSTM is correctly predicting the different student performance categories.

8. Final Architecture

Weekly Attendance + Assignment Marks + Internal Marks + Previous Exam Scores



Sequence of Weekly Data



LSTM Layer



Dropout



Dense Layer



Softmax



Low / Average / High Performance

Conclusion

An **LSTM-based model** is appropriate for predicting student performance because the input data is collected over multiple weeks and has a sequential relationship. The LSTM can remember important information from previous weeks and use it together with current academic information to predict the student's future performance category.

Q4. Bidirectional RNN for Sentiment Classification

Question:

An organization wants to classify customer reviews as **positive, negative, or neutral**. Since the meaning of a word may depend on both the words appearing before and after it, design a **Bidirectional RNN model** for this application. Identify the major processing stages from input text to final sentiment prediction.

Solution:

1. Problem Understanding

The objective is to classify customer reviews into three sentiment categories:

- **Positive**
- **Negative**
- **Neutral**

The meaning of a word can depend on both the **previous words and the following words** in a sentence. Therefore, a **Bidirectional RNN (BiRNN)** is suitable because it processes the text in both forward and backward directions.

2. Proposed Model Architecture

The model can be designed as:

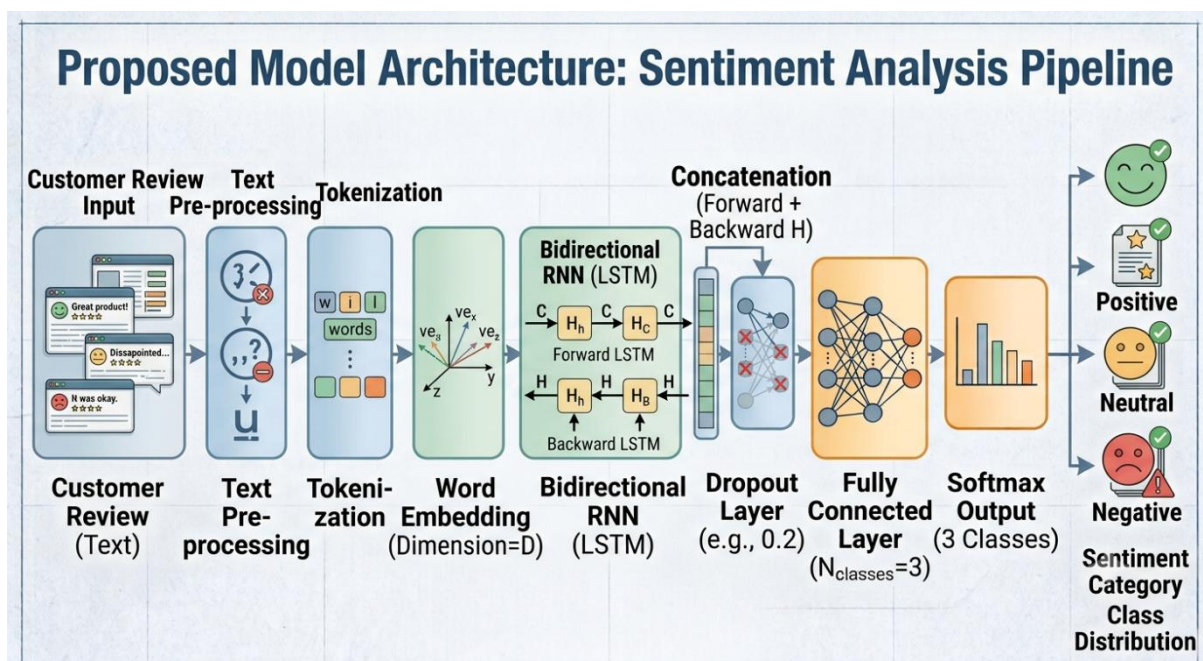


Figure 4: Proposed Architecture

3. Text Pre-processing

The customer review is first converted into a form that can be processed by the neural network.

The major steps include:

- Removing unnecessary symbols and noise.

- Converting text into tokens.
- Converting words into numerical representations.
- Padding sequences so that inputs have a consistent length.

For example:

“The product is very good”

is converted into a sequence of tokens and then into numerical vectors.

4. Word Embedding

The tokenized words are passed through an **embedding layer**.

The embedding layer converts each word into a dense numerical vector that represents useful information about the word.

For example:

“good” → numerical feature vector

These word vectors are then provided to the Bidirectional RNN.

5. Bidirectional RNN

A normal RNN processes a sentence in one direction:

Beginning → End

A Bidirectional RNN processes the sentence in two directions:

Forward RNN:

Beginning → End

Backward RNN:

End → Beginning

The outputs from both directions are combined.

This allows the model to use information from both the words **before and after** a particular word.

For example, in:

“The movie was not very good”

the model can use surrounding words to better understand the sentiment associated with the sentence.

6. Classification Layer

After processing the complete review, the forward and backward RNN representations are combined and passed to a **fully connected layer**.

The final layer uses **Softmax activation** with three output neurons:

Output	Sentiment
Class 1	Positive
Class 2	Negative

The class having the highest probability is selected as the final sentiment.

7. Training Strategy

The model can be trained using:

- A dataset containing customer reviews and their sentiment labels.
- Training and validation datasets.
- **Categorical cross-entropy loss** for three-class classification.
- **Adam optimizer**.
- Multiple training epochs.
- Dropout to reduce overfitting.

The model learns relationships between words and their surrounding context during training.

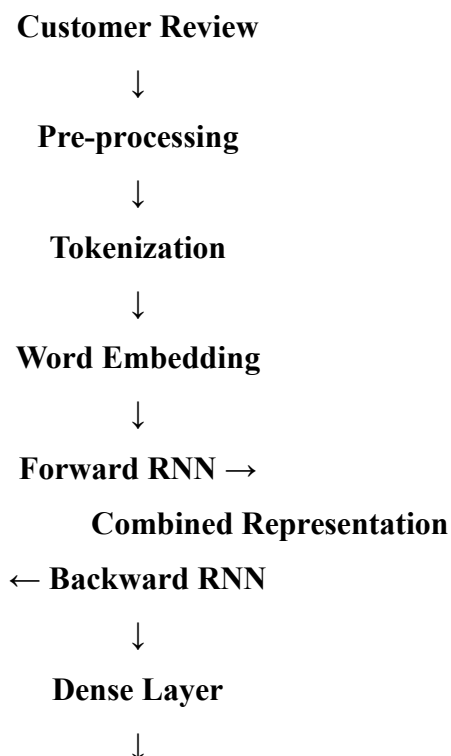
8. Performance Evaluation

The model can be evaluated using:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-score**
- **Confusion Matrix**

The confusion matrix is particularly useful for identifying whether the model is confusing positive, negative, and neutral reviews.

9. Final Architecture



Softmax (3 Classes)



Positive / Negative / Neutral

Conclusion

A **Bidirectional RNN** is suitable for sentiment classification because it considers information from both directions of a sentence. The review is first pre-processed and converted into word embeddings. The Bidirectional RNN then processes the sequence from both directions, combines the information, and passes it to a Softmax classification layer to predict whether the review is **positive, negative, or neutral**.

Q5. Encoder–Decoder Model for English-to-Hindi Translation

Question:

A software company wants to develop a system that translates **English sentences into Hindi**. The length of the input and output sentences may differ, and the system needs to understand the complete input sentence before generating the translated sentence. Design an **Encoder–Decoder model** and describe the role of the encoder, decoder, hidden representation, and output layer in the translation process.

1. Problem Understanding

The objective is to translate an English sentence into Hindi using a neural network.

For example:

English: “I am going to school.”

Hindi: “मैं स्कूल जा रहा हूँ।”

The English and Hindi sentences may have **different lengths**, so a fixed one-to-one mapping between input and output words is not suitable.

An **Encoder–Decoder architecture** can solve this problem by first understanding the complete English sentence and then generating the Hindi translation.

2. Proposed Model Architecture

The proposed architecture is:

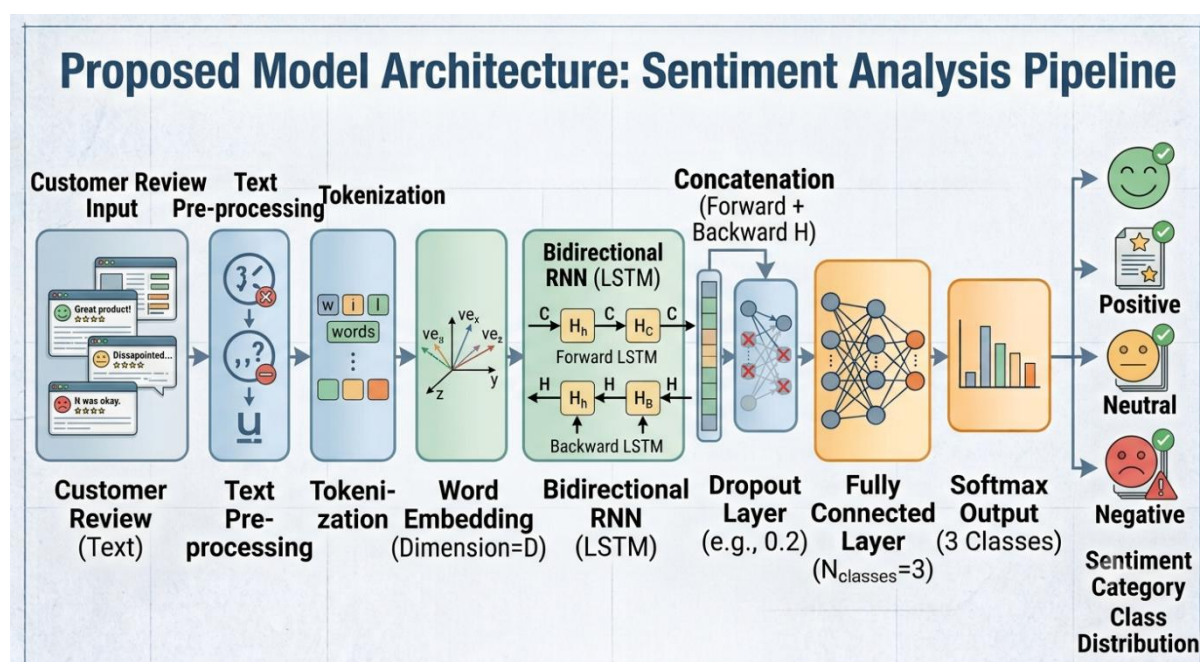


Figure 5: Proposed Architecture

3. Encoder

The **encoder** receives the English sentence one word at a time.

For example:

“I am going to school”

The words are converted into numerical vectors using an embedding layer and then processed sequentially by the encoder.

The encoder captures information about the input sentence and produces a **hidden representation** containing the important information needed for translation.

4. Hidden Representation

The hidden representation acts as a summary of the information learned from the English input sentence.

It carries information about:

- The words in the sentence
- Their order
- Relationships between words
- Overall meaning of the sentence

This representation is then passed to the decoder.

5. Decoder

The **decoder** uses the hidden representation generated by the encoder to produce the Hindi sentence.

It generates the translation **one word at a time**.

For example:

<START> → मैं → स्कूल → जा → रहा → हूँ → <END>

At each step, the decoder uses the information from the previous step and the encoded representation to predict the next Hindi word.

6. Output Layer

The decoder output is passed to a **Softmax output layer**.

The Softmax layer gives probabilities for possible Hindi words in the vocabulary.

For example:

Hindi Word	Probability
मैं	0.80
वह	0.10
हम	0.05
यह	0.05

The word with the highest probability can be selected as the next output word.

This process continues until the decoder generates an **end-of-sentence token**.

7. Training Strategy

The model can be trained using a dataset containing pairs of:

English sentence → Hindi sentence

During training:

- English sentences are given to the encoder.
- Corresponding Hindi sentences are provided as target outputs.
- The decoder learns to predict the next Hindi word.
- **Categorical cross-entropy loss** can be used.
- An optimizer such as **Adam** can be used.
- Training data can be divided into training and validation sets.

8. Handling Different Sentence Lengths

One important advantage of the Encoder–Decoder model is that the input and output sequences do not need to have the same length.

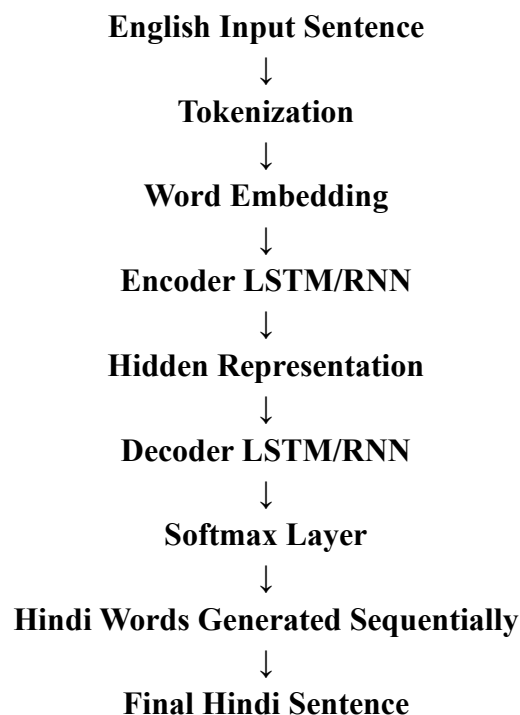
For example:

English: “How are you?”

Hindi: “आप कैसे हैं?”

The number of words can differ, but the decoder can continue generating words until the complete Hindi sentence is produced.

9. Final Architecture



10. Role of Each Component

Component	Role
Encoder	Processes the complete English input sentence
Hidden Representation	Stores important information about the input sentence
Decoder	Generates the Hindi translation sequentially
Softmax Output Layer	Predicts the probability of the next Hindi word

Conclusion

An **Encoder–Decoder model** is suitable for English-to-Hindi translation because it can handle input and output sentences of different lengths. The encoder processes and represents the English sentence, the hidden representation transfers the learned information to the decoder, and the decoder generates the Hindi translation one word at a time. The Softmax output layer selects the most probable Hindi word at each step

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Requirement Analysis	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding ; some requirements unclear	Poor understanding ; misinterprets problem context
Model Architecture & Design	30%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Application of Deep Learning Techniques	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Model Development & Implementation Strategy	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Performance Analysis & Justification	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis